Compte rendu TP – Carte sans-contact MIFARE Classic

Ben Aissa Yassine · Elammous Abdelmmouheimen

1. Introduction

L'objectif de ce TP était de développer une application avec interface graphique sous Qt pour gérer un porte-monnaie électronique sur une carte sans contact MIFARE Classic 1K. Le projet s'inscrit dans l'idée d'une carte multiservice pour les étudiants de l'ESIREM, qui contiendrait l'identité de l'étudiant et un crédit d'unités pour la cafétéria ou les photocopies.

L'application doit permettre de lire et d'écrire le nom et le prénom sur la carte, de débiter et de recharger des unités en gérant un backup. Le lecteur doit aussi donner une information visuelle et sonore à chaque transaction.

Pour ce TP nous avons utilisé le matériel et les logiciels suivants :

Élément	Détail
Carte	NXP MIFARE Classic 1K (MF1S50YYX)
Lecteur	Coupleur ODALID USB (CDC)
Librairie	ODALID Education (`ODALID_Education.dll`, `libODALID_Education.a` et les headers du dossier `LIB`)
Environnement	Qt Creator, kit MinGW 64-bit, C++17

2. Architecture de la carte

Nous avons repris l'architecture définie dans la partie 2.2 du sujet, qui est commune à toutes les applications. L'identité est stockée dans le secteur 2 et le porte-monnaie dans le secteur 3.

Secteur	Bloc	Contenu
2	8	« Identite » (nom de l'application)
2	9	Prénom
2	10	Nom
2	11	Clé A + bits d'accès + clé B
3	12	« Porte Monnaie » (nom de l'application)
3	13	Backup du compteur

Secteur	Bloc	Contenu
3	14	Compteur
3	15	Clé A + bits d'accès + clé B

Un bloc fait 16 octets, donc le nom et le prénom font au maximum 16 caractères. Les blocs 13 et 14 sont au format « value block », ce qui permet d'utiliser directement les commandes d'incrément et de décrément de la carte.

Pour l'authentification, les clés sont stockées dans le Secure Element du lecteur. Dans le programme, on donne seulement l'index de la clé et on laisse les fonctions de la librairie faire l'authentification automatiquement (paramètre `Auth` à `TRUE`). L'avantage est que les clés n'apparaissent jamais dans le code.

Opération	Secteur	Clé	Index
Lire l'identité	2	A	2
Écrire l'identité	2	B	2
Lire / décrémenter le compteur	3	A	3
Écrire / incrémenter le compteur	3	B	3

3. Architecture de l'application

L'interface a été réalisée avec Qt Designer : la fenêtre est décrite dans le fichier mainwindow.ui, puis chargée dans le code avec ui->setupUi(this). Les widgets ont des noms explicites (btnConnect, lineNom, spinDec, labelUnits…), ce qui rend le code plus lisible. Au lancement, l'application force aussi le thème sombre de Qt (à partir de Qt 6.8). Le résultat est le suivant :

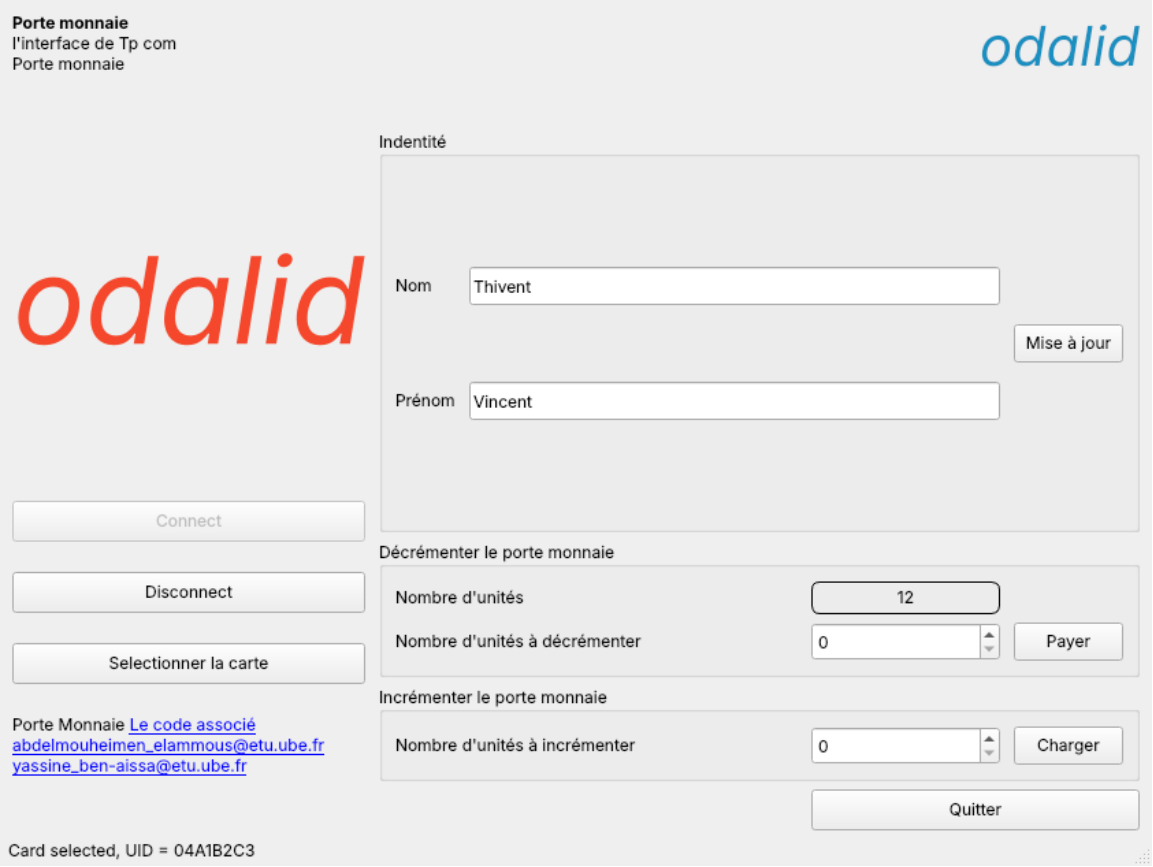


Figure 1 : interface de l'application (mainwindow.ui)

Pour ne pas mélanger l'interface graphique et les appels à la librairie, nous avons séparé le programme en plusieurs classes. La fenêtre (`MainWindow`) passe par une classe abstraite `CardBackend` qui contient une méthode pour chaque action sur la carte. Cette classe a deux implémentations :

- `OdalidBackend`, qui utilise le vrai lecteur avec la librairie ODALID ;
- `SimBackend`, qui simule une carte en mémoire. Elle nous permet de tester l'interface sans le lecteur.

Figure 2 : organisation du programme

Le projet contient les fichiers suivants :

Fichier	Rôle
main.cpp	Lancement de l'application
mainwindow.ui	Description de la fenêtre, créée avec Qt Designer
mainwindow.h/.cpp	Logique de l'interface et les 6 séquences de

Fichier	Rôle
	la partie 2.3
cardbackend.h	Classe abstraite, une méthode par action
odalidbackend.h/.cpp	Appels à la librairie ODALID
simbackend.h/.cpp	Carte simulée pour les tests
PorteMonnaie.pro	Fichier projet qmake (liaison avec ODALID_Education et ws2_32, copie de la DLL)

4. Réalisation des fonctionnalités

Pour chaque bouton, nous avons suivi le diagramme de séquences correspondant dans le sujet. Le tableau suivant résume les fonctions de la librairie appelées.

Partie	Bouton	Fonctions de la librairie appelées
2.3.1	Connect	GetLibraryExtension, OpenCOM, Version, RF_Power_Control(TRUE)
2.3.2	Sélectionner la carte	ISO14443_3_A_PollCard, Mf_Classic_Read_Block (blocs 10 et 9), Mf_Classic_Read_Value (bloc 14)
2.3.3	Mise à jour	Mf_Classic_Write_Block (blocs 10 et 9), puis Mf_Classic_Read_Block
2.3.4	Payer	Mf_Classic_Decrement_Value, Mf_Classic_Restore_Value, Mf_Classic_Read_Value
2.3.5	Charger	Mf_Classic_Increment_Value, Mf_Classic_Restore_Value, Mf_Classic_Read_Value
2.3.6	Disconnect	RF_Power_Control(FALSE), CloseCOM

4.1 Connexion

Pour la connexion, on initialise la structure ReaderName en mode ReaderCDC (lecteur USB) avec device = 10, qui correspond au port COM10 sur lequel le lecteur est reconnu par

Windows. On ouvre ensuite la communication avec OpenCOM(), on lit la version du lecteur et on active le champ RF. Si tout se passe bien, la LED du lecteur passe au vert.

Si le lecteur ne répond pas, la fonction OpenCOM() peut rester bloquée plusieurs secondes, ce qui figerait toute la fenêtre. Nous avons donc lancé la connexion dans un thread séparé avec QThread::create ; à la fin, le résultat est renvoyé à la fenêtre avec QMetaObject::invokeMethod. Si le lecteur n'a toujours pas répondu au bout de 10 secondes, un message indique les vérifications à faire : port COM visible dans le Gestionnaire de périphériques, aucun autre programme qui utilise le lecteur, débrancher puis rebrancher le lecteur.

Des messages qDebug() affichent aussi chaque étape de la connexion dans la console (avant et après OpenCOM, Version et RF_Power_Control), avec le code de retour de chaque fonction.

4.2 Identité

La lecture du nom et du prénom se fait avec la clé A et l'écriture avec la clé B (index 2). Comme un bloc fait 16 octets, nous avons limité les champs Nom et Prénom à 16 caractères, et le texte est complété par des zéros avant l'écriture. Après une mise à jour, on relit les deux blocs pour vérifier ce qui a été écrit sur la carte.

4.3 Paiement

Avant de débiter, on vérifie que le nombre d'unités demandé ne dépasse pas le solde. Si c'est le cas, on affiche « Insufficient balance » et rien n'est envoyé à la carte. Sinon on appelle les deux fonctions suivantes (clé A, index 3) :

```
int16_t st = Mf_Classic_Decrement_Value(&reader, TRUE, 14, n, 13, AuthKeyA, 3);
if (st != MI_OK) return fail(st, "Mf_Classic_Decrement_Value");
st = Mf_Classic_Restore_Value(&reader, TRUE, 13, 14, AuthKeyA, 3);
```

4.4 Rechargement

Le diagramme 2.3.5 du sujet semble être une copie de celui du paiement : on y voit le message « Payer » et la fonction `Mf_Classic_Decrement_Value`. Pour recharger, nous avons donc utilisé `Mf_Classic_Increment_Value` avec la clé B, ce qui correspond aux droits définis pour le compteur. Le reste de la séquence est identique au paiement.

5. Backup, signalisation et erreurs

5.1 Gestion du backup

Pour ne pas perdre de crédit si la carte est retirée pendant une opération, nous utilisons le bloc 13 comme backup du compteur. Un paiement se fait en deux étapes :

1. on décrémente le bloc 14 et le résultat est transféré dans le bloc 13, le bloc 14 garde donc encore l'ancienne valeur ;
2. on fait un Restore du bloc 13 et on le transfère dans le bloc 14.

Le rechargement fonctionne de la même façon avec un incrément. Si la carte sort du champ entre les deux étapes, il reste toujours une valeur correcte dans l'un des deux blocs.

5.2 Information visuelle et sonore

Après chaque transaction, le programme appelle la fonction `LEDBuzzer()` du lecteur :

Cas	LED	Buzzer
Lecteur prêt	Verte	aucun
Transaction réussie	Verte	bip court (100 ms)
Erreur	Rouge	bip long (400 ms)

5.3 Gestion des erreurs

Toutes les fonctions de la librairie renvoient un code de retour. S'il est différent de `MI_OK`, on récupère le message correspondant avec `GetErrorMessage()` et on l'affiche dans une fenêtre et dans la barre d'état. Après une erreur d'authentification, la carte repasse dans l'état IDLE/HALT, c'est pourquoi on refait un `ISO14443_3_A_PollCard()` pour pouvoir continuer à l'utiliser.

Les boutons sont aussi activés ou désactivés selon l'état de l'application : au départ seul Connect est actif, puis Disconnect et Sélectionner la carte après la connexion, et enfin les parties Identité, Décrémenter et Incrémenter une fois la carte sélectionnée.

6. Difficultés

La première difficulté a été la compilation : la librairie ODALID est compilée avec MinGW 64-bit, il faut donc utiliser le kit Qt MinGW 64-bit et pas MSVC. Il faut aussi lier la librairie `ws2_32` et placer la DLL à côté de l'exécutable. Nous avons ajouté une copie automatique de la DLL dans le fichier `.pro`.

Nous avons aussi eu des difficultés pour nous connecter au lecteur. Il faut indiquer dans le champ device de ReaderName le numéro du port COM du lecteur (10 dans notre cas). Pour trouver où la connexion bloquait, nous avons ajouté des messages qDebug() à chaque étape, puis nous avons déplacé la connexion dans un thread pour que l'interface ne se fige plus.

Il faut aussi que les cartes soient bien préparées. Les blocs 13 et 14 doivent être au format value block, sinon les lectures et les opérations sur le compteur échouent ; on peut les initialiser avec `Mf_Classic_Write_Value()`. De même, les clés rangées aux index 2 et 3 du lecteur doivent correspondre à celles de la carte, sinon l'authentification ne passe pas.

Enfin, on pourrait ajouter une partie administration pour personnaliser les cartes : chargement des clés avec `Mf_Classic_LoadKey()`, écriture des conditions d'accès avec `Mf_Classic_UpdadeAccessBlock()` et initialisation des compteurs.

7. Conclusion

Ce TP nous a permis de mettre en pratique le fonctionnement d'une carte MIFARE Classic avec un vrai lecteur et une librairie industrielle. L'application réalise les fonctions demandées : lecture et écriture de l'identité, paiement et rechargement avec un backup, et signalisation par LED et buzzer à chaque transaction. L'interface a été faite avec Qt Designer et nous avons suivi les six diagrammes de séquences du sujet et l'architecture de carte de la partie 2.2.

Nous avons aussi compris l'intérêt de séparer les droits entre les deux clés : la clé A suffit pour lire et payer, mais il faut la clé B pour écrire et recharger. Garder les clés dans le Secure Element du lecteur évite de les mettre dans le programme.